

Coding & Solution

Date	July 3, 2026
Team ID	xxxxxx
Project Name	Credit Card Approval Prediction System
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/aliasuhana2005-lab/Credit-Card-Approval-Prediction-System
Programming Language(s)	Python (model training & backend), HTML, CSS, JavaScript (frontend)
Framework(s) Used	Flask, scikit-learn, joblib, pandas, NumPy, Matplotlib, Seaborn
Key Features Implemented	- Data exploration notebook (CreditCard.ipynb): loads creditcard.csv, performs univariate and multivariate analysis, removes duplicates, checks for missing values, and engineers Log_Amount and Hour features. - Trained and compared Logistic Regression, Decision Tree, and Random Forest classifiers; the Random Forest model was selected and saved as credit_card_model.pkl using joblib. - Flask backend (app.py) that loads the trained model and exposes a /predict route (GET/POST) with try/except error handling around the prediction step. - Front-end UI (index.html) built with Tailwind CSS in a glass-morphism style, plus a separate style.css for a classic card-based form layout.

Pending / Incomplete Features	- index.html currently runs a client-side threshold check in JavaScript rather than submitting the form to the Flask /predict endpoint, so the trained model is not yet wired to this UI. - The predict.html and result.html templates referenced in app.py are not yet included. - The current form fields (income type, employment duration, income level, credit score) do not yet map to the model's actual training features (Time, Amount, V1-V28, engineered Log_Amount/Hour); this alignment still needs to be implemented. - No feature scaling or class-imbalance handling (e.g., SMOTE) has been applied yet, despite the dataset being highly imbalanced.
Setup / Run Instructions	1. Install dependencies: <code>pip install flask joblib numpy pandas scikit-learn</code> 2. Place <code>credit_card_model.pkl</code> in the project root alongside <code>app.py</code>. 3. Run the app: <code>python app.py</code> 4. Open <code>http://127.0.0.1:5000/</code> in a browser to view the UI.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	No

Additional Notes / Comments:

The notebook (CreditCard.ipynb) trains and compares Logistic Regression, Decision Tree, and Random Forest classifiers on the creditcard.csv dataset, and saves the Random Forest model as credit_card_model.pkl. The Flask backend (app.py) loads this model and exposes a /predict route with try/except error handling. The current index.html uses a Tailwind-based UI whose client-side logic is not yet connected to the Flask backend, and its input fields do not match the model's trained features. Wiring the form to /predict with matching features, adding the missing predict.html/result.html templates, and addressing class imbalance in the dataset are recommended next steps before final submission.